AI 協作反思報告

專案名稱：校園二手交易平台　｜　作者：許家羚（114153016）

一、 我怎麼使用 AI

在開發的過程中，我主要透過 Google AI Studio 進行畫面與功能實作，並搭配 Claude 梳理規格文件、設計結構化提示詞與規劃測試驗證表。我的原則是「分階段推進，每次只聚焦解決一個問題」，絕不一次性拋出全站需求；每項功能實作完成且手動驗證無誤後，才進入下一個環節。在進行除錯時，我固定在提示詞中明確規範三項要素：當前的異常現象、預期的正確結果，以及「嚴格限縮修改範圍，不得變動其他既有模組」。

【代表性提示詞範例】
「保留目前專案的所有 *UI/UX* 排版、元件設計與視覺規範，只將底層資料存取全面改為純前端 *Firebase Firestore Client SDK* 直連。
在前端使用 *onSnapshot* 即時監聽 *items* 集合，若為空則自動載入示範資料。請嚴格維持現有全部畫面動線與元件結構，僅更換資料存取邏輯，不要改動其他無關功能。」

二、 AI 建議的問題與修正過程

• **問題 1：AI 的自動測試報告不可信：**AI 宣稱 91 項測試全數通過，但實測發現後端資料庫根本未成功連線，畫面僅讀取寫死的假資料；我果斷改用 Firebase 直連並以即時監聽同步資料，體會到 AI 容易把「程式碼有寫」誤報為「功能已通過」，一切必須親自驗證。

• **問題 2：登入功能一開始就無法使用：**註冊時因 Firebase 預設停用 Email 登入而報錯，AI 無法替我操作外部後台；我親自前往控制台啟用服務並補齊網域授權，學到外部服務的設定必須由開發者親自把關。

• **問題 3：圖片過大導致資料庫寫入失敗：**上傳未壓縮的大圖轉成 Base64 後超過 Firestore 單一文件 1MB 上限；我在前端實作 Canvas 壓縮技術將照片壓至 100KB 內解決問題，深刻理解到掌握架構限制才能評估 AI 解法的可行性。

• **問題 4：示範商品圖片與標題不符：**初期隨機圖庫導致商品圖文脫節，且批次修復時常誤改自訂品項；我針對 24 筆商品逐一精準配圖並加入防破圖機制，學會向 AI 下指令時必須嚴格界定「哪些資料不可改動」。

• **問題 5：UI 雜亂且缺乏一致性：**初期介面顏色過多、四格身分按鈕佔據大量版面且有重複搜尋框；我以學院徽章 Logo 的深藏青為主色收斂全站視覺，並將測試身分濃縮至右上角選單，使平台符合校園市集的沉穩氣質。

• **問題 6：權限規則為了展示而放寬：**為了讓評審測試不被阻擋，安全規則暫時設為全開；我清楚這是展示期的權衡，因此堅持在文件已知限制中誠實揭露，絕不粉飾系統安全性。

三、 哪些判斷由我自己完成

• 決定功能範圍與優先級，其餘誠實標為模擬或未完成

• 不盲信 AI 的「全部通過」報告，堅持用多帳號端對端實測

• 主動追問資料存儲真相，識破假資料並果斷更換為 Firestore 架構

• 在點數有限時果斷停損，回滾並凍結穩定版本以備錄影

• 排定修復優先級，堅持「流程走通與資料落地」重於視覺美化

• 誠實揭露安全規則全開、Base64 存圖限制等已知缺陷

• 主導 UI 風格收斂，依據學院 Logo 重新規劃色彩與簡化動線

• 主動將開發預覽網址更換為公開網址，避免評審因 Google 登入驗證受阻

【為什麼這些要由我自己判斷】

因為 AI 只是執行程式碼生成的輔助工具，它無法同理真實使用者的操作體驗，更無法替專案的最終交付品質承擔責任。不論是底層架構的推翻重建、安全風險的評估、開發點數耗盡時的取捨，還是面對系統缺陷時的誠實揭露，這些核心決策都涉及工程判斷與責任感，必須由開發者全權把關，才能打造出真正可用、可信賴的產品。

四、 個人學習心得

這趟 Vibe Coding 的實踐讓我體會到，現代 AI 開發的真正瓶頸不是「生成程式碼」，而是開發者的「驗證能力與工程批判思維」。AI 能在短時間內產出看似完整的介面與自信滿滿的報告，但背後常隱藏著假資料與設定斷層；唯有透過具體分段的提示詞、追問底層資料流向，並親自以多角色實測，才能把生成的半成品轉化為真正運行的系統。此外，在有限的點數與時間內進行風險管理、排定修復優先級，更是至關重要的實戰經驗。最重要的是我學會了「誠實」——在技術文件裡清楚標明架構限制、展示權衡與未完成項目，遠比粉飾出一份虛假的完美成品，展現出更成熟且負責任的工程師素養。